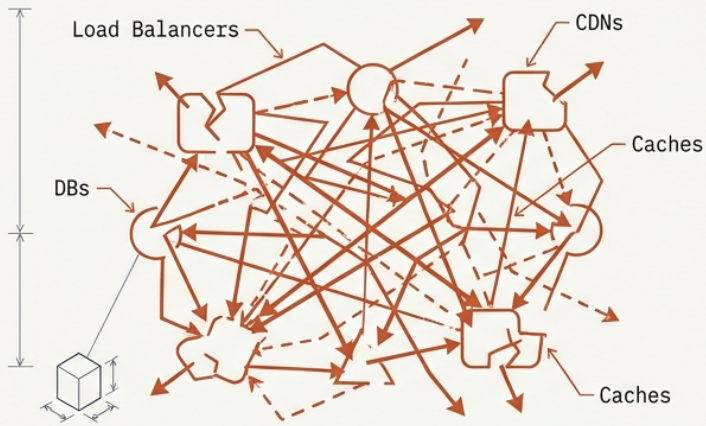


The 45-Minute System Design Architect

A Framework for Mastery.

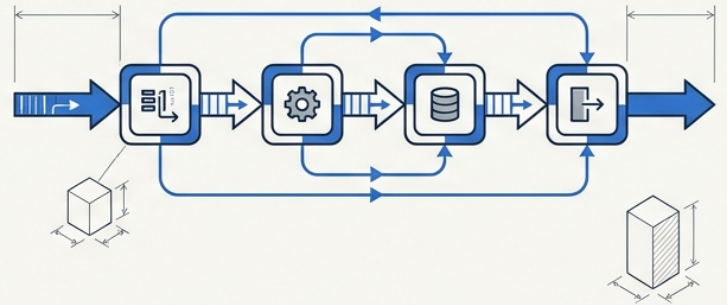
AD HOC APPROACH (CHAOS)



Thousands of components. Only 45 minutes.

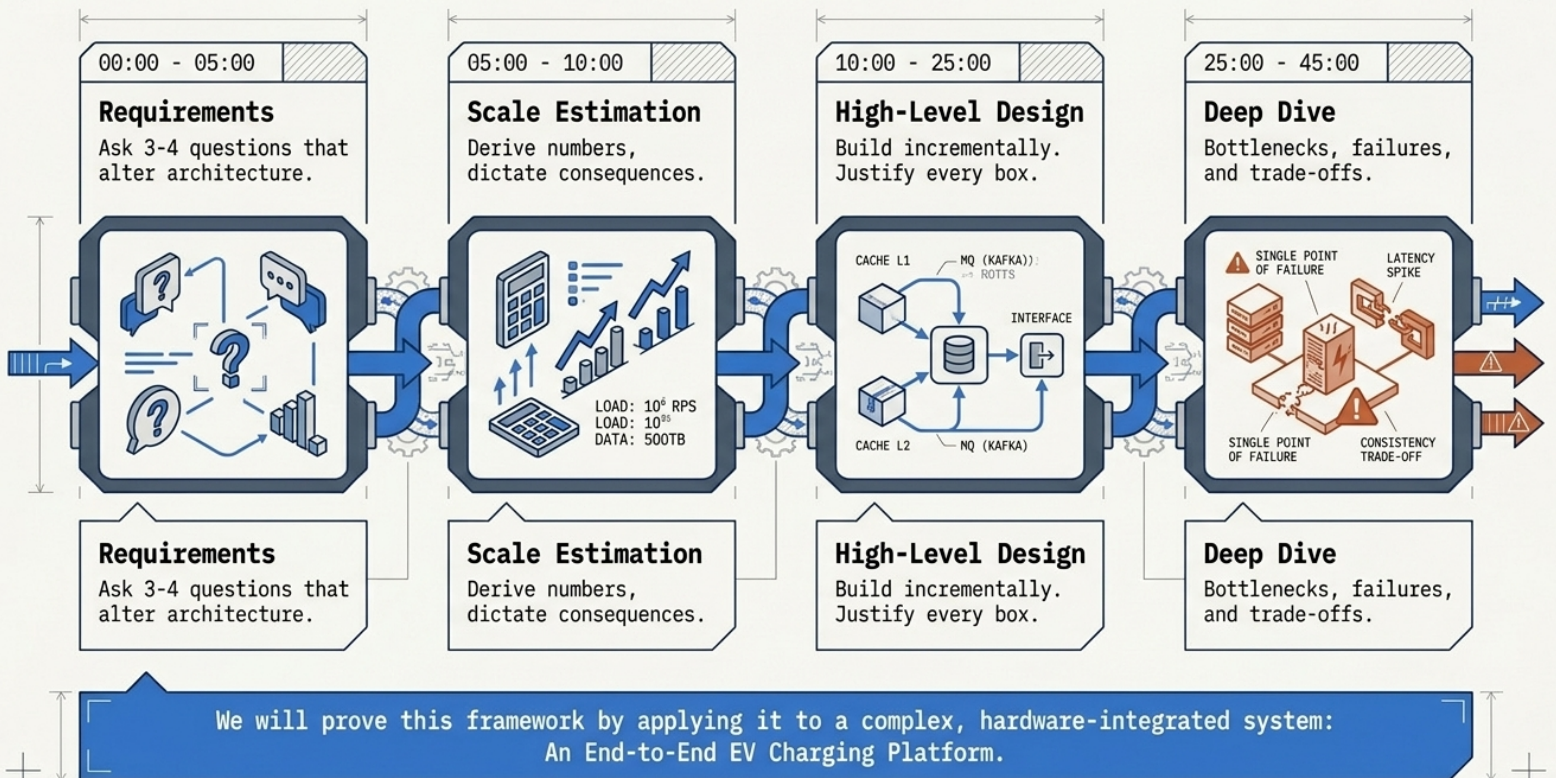
- Skip scale
- Jump to DBs
- Over-engineer
- Run out of time

STRUCTURED APPROACH (ORDER)



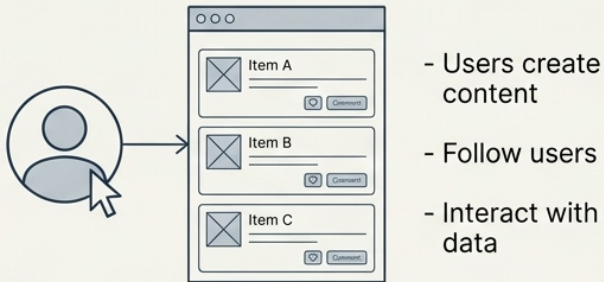
Structure over memorization.

- Structure ambiguity
- Estimate scale
- Defend trade-offs
- Identify bottlenecks



Phase 1: The Boundary of Requirements

Functional: The “What”



Non-Functional: The “How Well”



Ask only the questions that materially alter the architecture. Limit to 3-4 core journeys.

The Geographic Multiplier



User location fundamentally **rewrites your data layer.**

Case Study: Scoping the EV Platform

Functional Scope

-  **Discovery:**
Find nearby stations
-  **Reservation:**
Lock a connector
-  **Charging:**
Start/Stop telemetry
-  **Payment:**
Billing execution

Non-Functional Targets

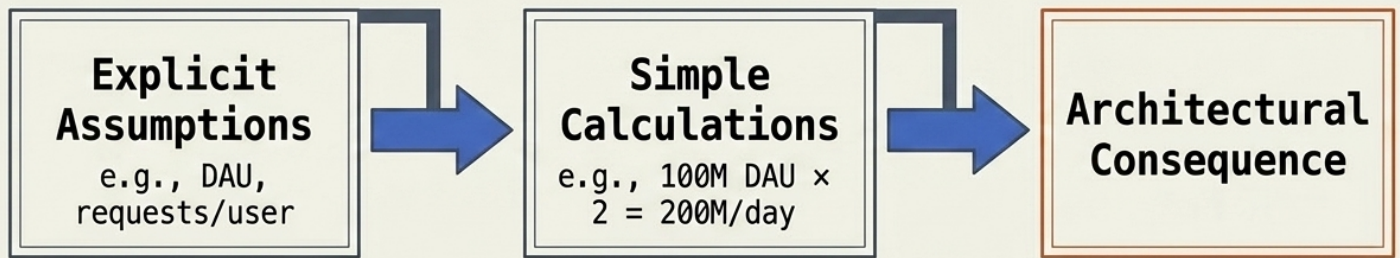
AVAILABILITY : **99.99%**
(Core Platform)

CONSISTENCY : **Strong**
(Money/Reservations)

CONSISTENCY : **Eventual**
(Search/Analytics)

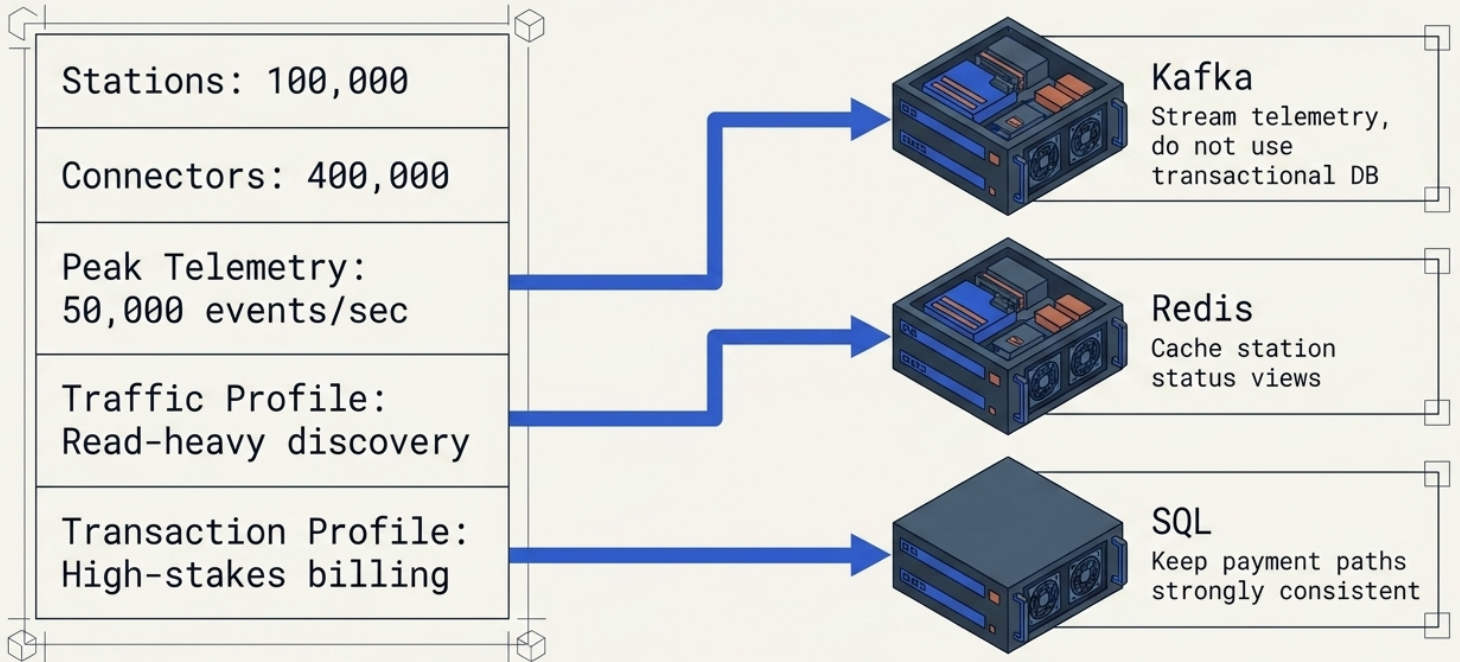
LATENCY : **p95 < 500ms**
(Discovery)

Phase 2: The Logic of Scale

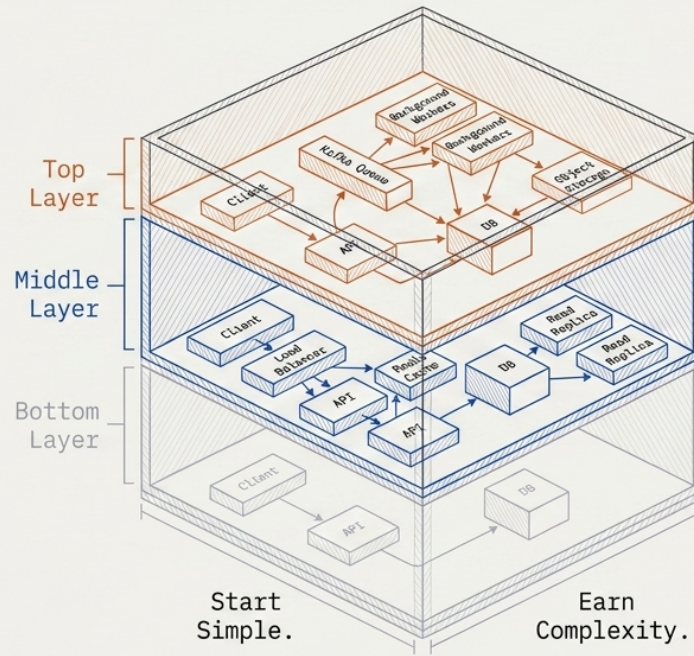


	Scale Trigger		System Design Output
1.	High Read QPS	→	Caching / CDN
2.	High Write QPS	→	Partitioning / Async Processing
3.	Large Data Volume	→	Distributed Storage

EV Scale: Architecting by the Numbers



Phase 3: The Incremental Build



DIAGNOSTIC CHECK

Do not blindly include every component. Every box must have a justification. What problem does it solve? What if it fails?

API Design: The 5-API Rule

POST /reservations

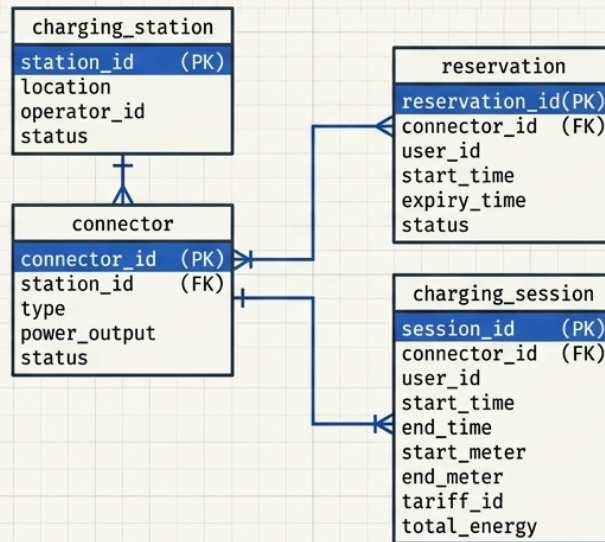
```
Request: {  
  "connector_id": "C123",  
  "user_id": "U456"  
}  
Response: {  
  "reservation_id": "R789",  
  "expiry": "10:05Z"  
}
```

POST /payments

```
Header:  
Idempotency-Key: SESSION-S123-FINAL  
  
Request: {  
  "session_id": "S123",  
  "amount": 14.50  
}
```

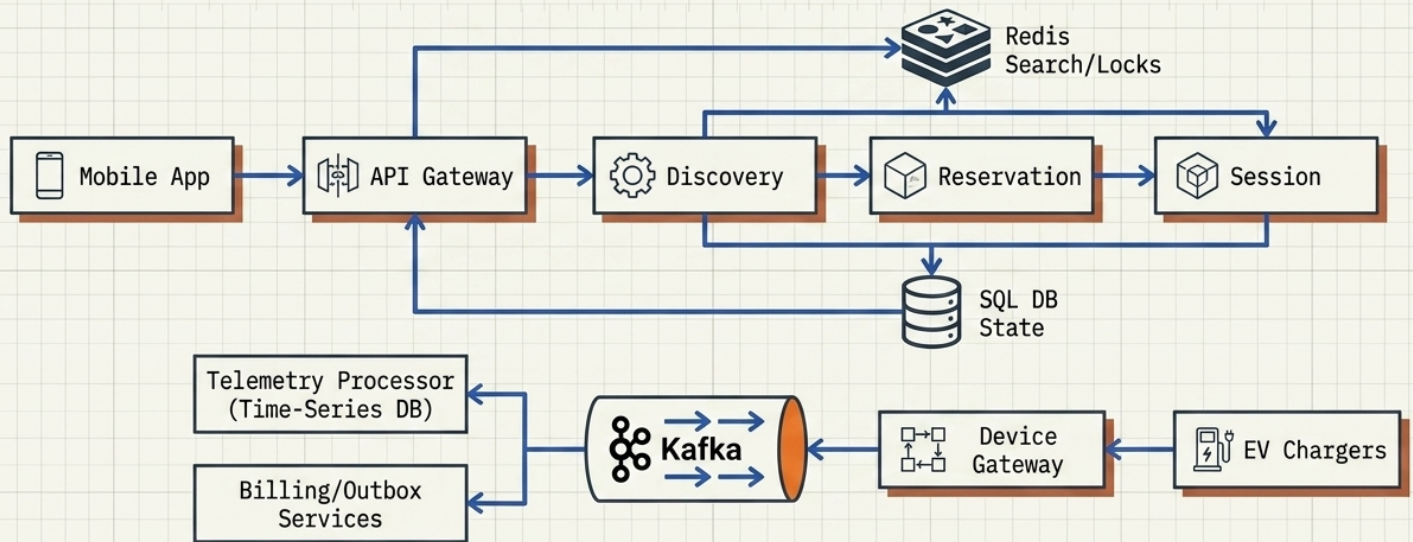
Keep APIs focused on core journeys. Always define idempotency considerations for mutating actions (like payments).

Relational Blueprints



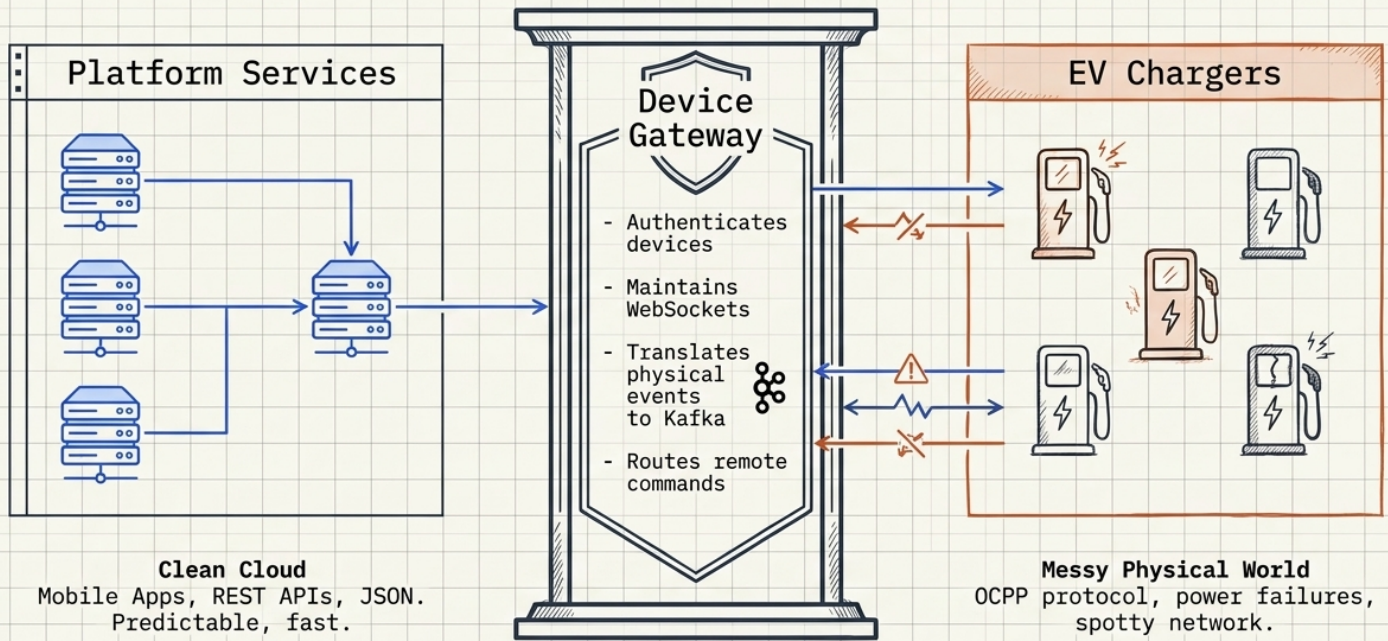
Define Primary Keys, access patterns, and relationships. Separate the intent (reservation) from the reality (charging_session).

The EV Platform Blueprint



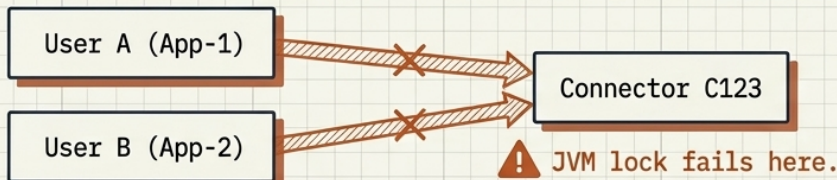
Redis coordinates. SQL protects correctness.
Kafka handles volume. Gateways isolate hardware.

Phase 4: The Hardware Divide

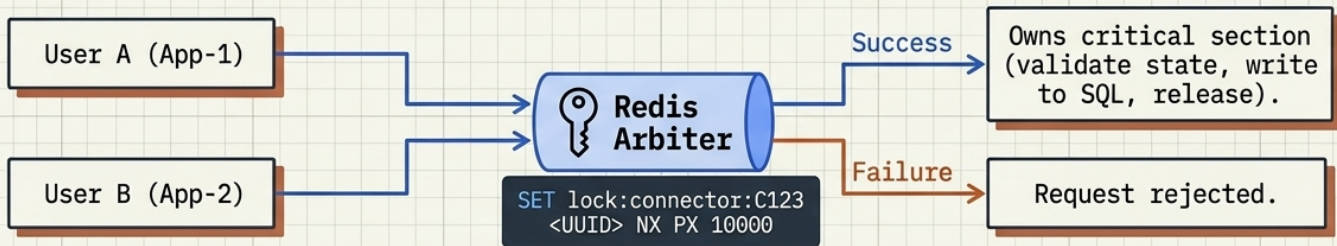


Concurrency & Distributed Locks

The Problem

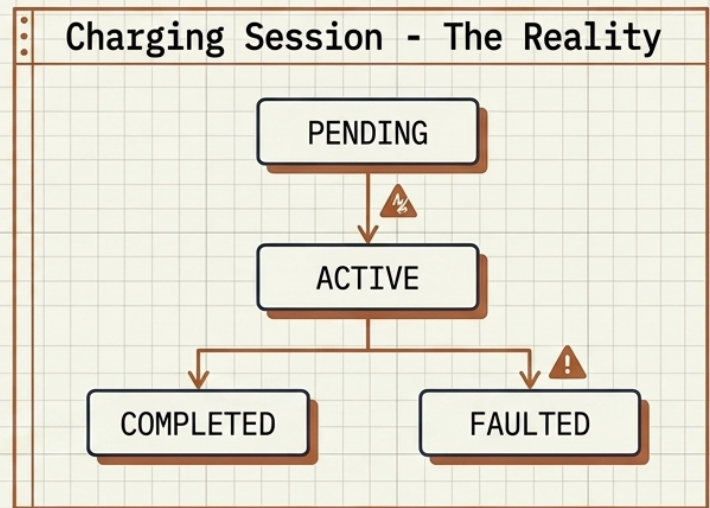
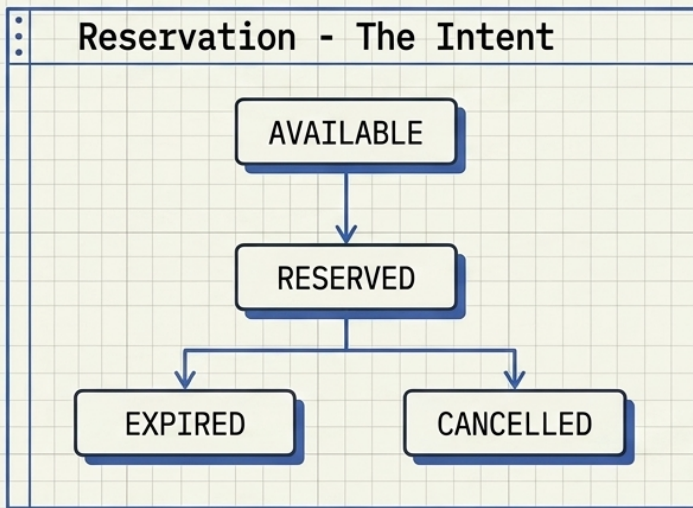


The Solution



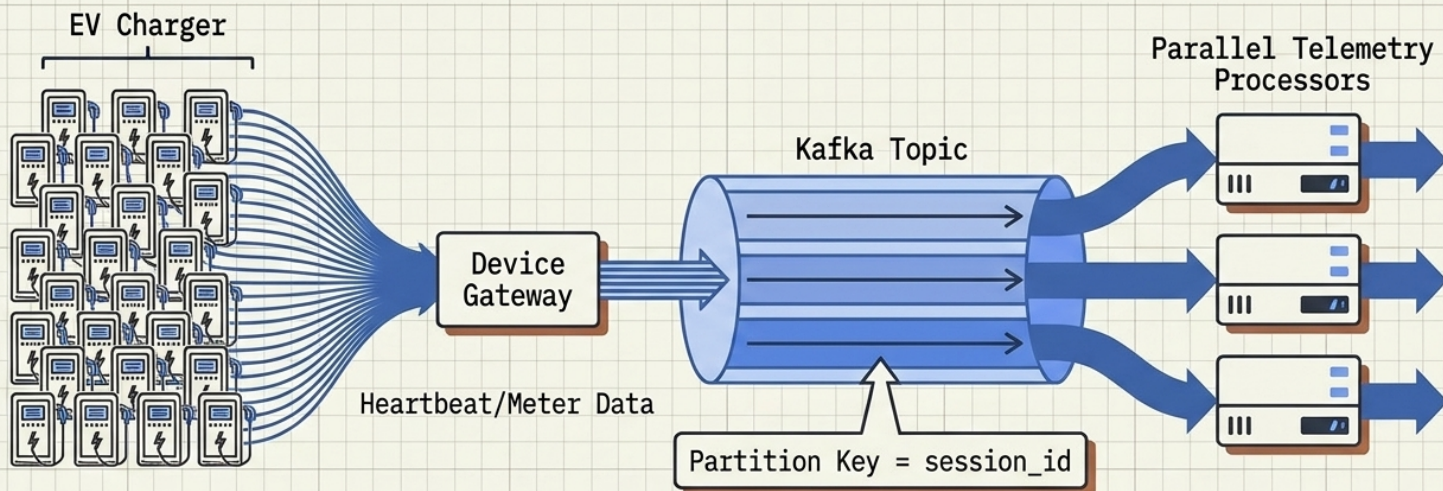
Never hold a distributed lock while waiting for a third-party payment.

State Machines: Intent vs. Reality



Accepting an API command is not proof that physics happened.
Keep reservation state separate from actual charging-session state.

Taming the 50k/sec Firehose



Telemetry overloads transactional DBs.
Same key -> same partition -> ordered processing.
Different keys -> parallel scaling.

Distributed Transactions: The Outbox Pattern

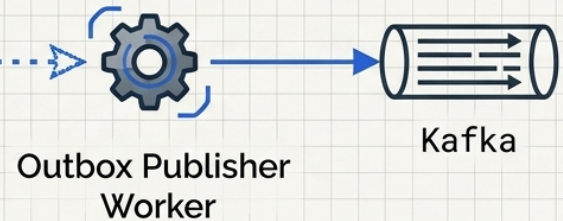
SQL Transaction Block

Step 1: Update session → COMPLETED

Step 2: Create invoice

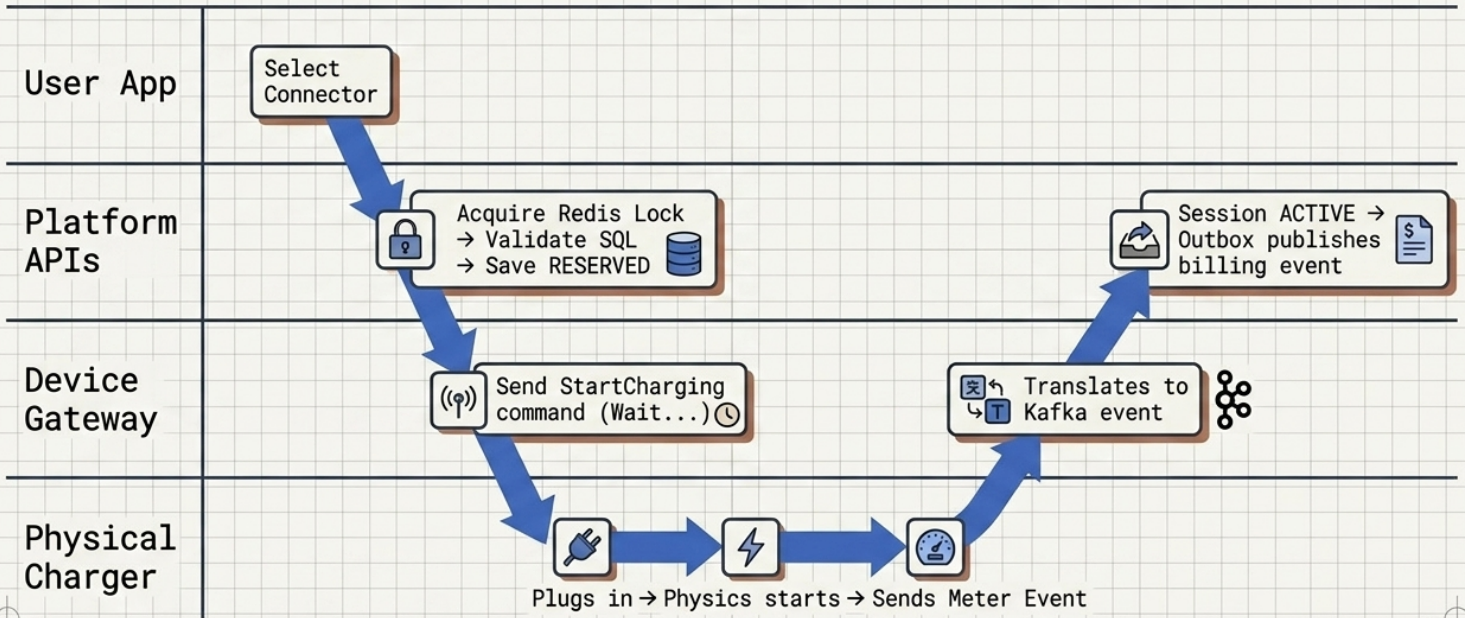
Step 3: Insert event into outbox_event table

COMMIT



Prevents the database from recording a completed session while the notification event disappears due to Kafka downtime.

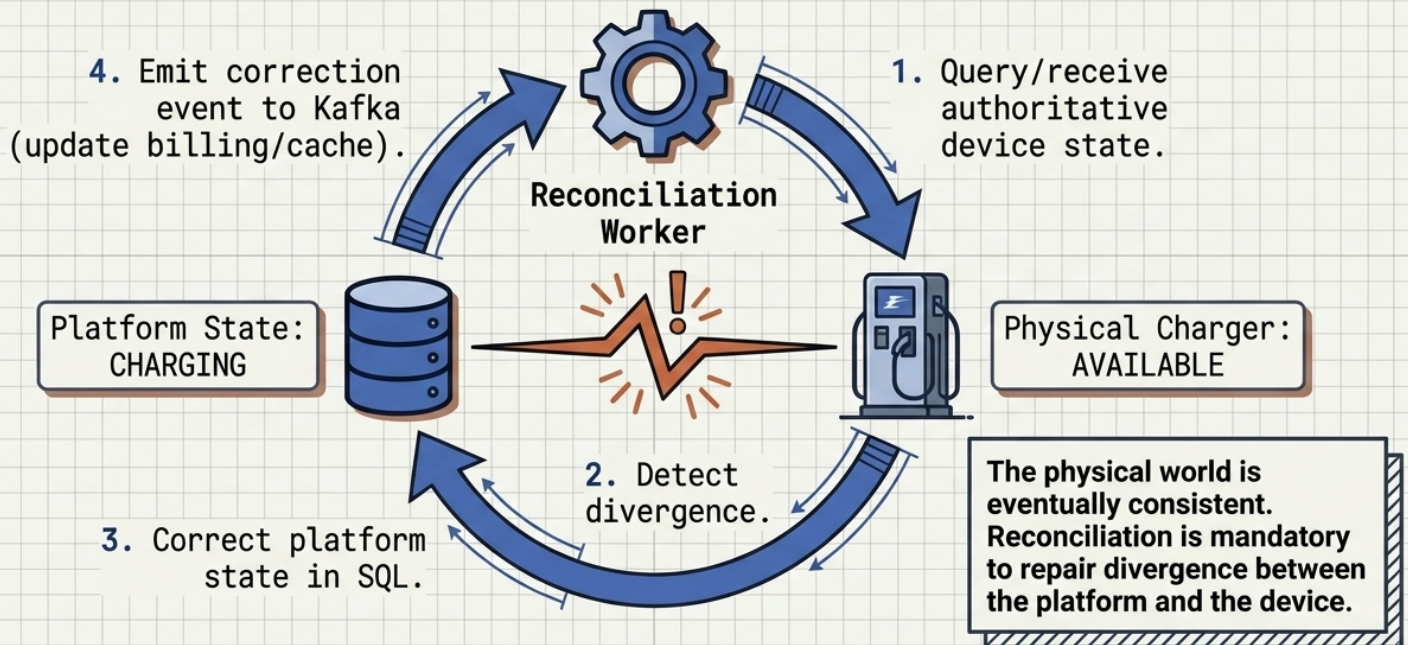
The End-to-End Orchestration



Failure Scenario Diagnostic

Failure Node 	Architectural Defense 
Network Partition (Charger offline)	Persist state locally; reconcile upon reconnect.
Duplicate Meter Event	Deduplicate via sequence numbers / Idempotent consumers.
Payment Gateway Timeout	Query provider using the exact same Idempotency-Key.
Redis Unavailable	Fail closed for reservations; SQL remains correctness boundary.

The Core Concept: Reconciliation



The Architect's Mindset

1. Physical Reality is Eventual

Accept that hardware and software will disagree.

2. Separate Intent from Reality

A successful API command is not proof of execution.

3. Idempotency is Non-Negotiable

Duplicates happen. Retries must be safe.

4. Reconcile Relentlessly

Build background workers that heal state divergence.

5. Justify Every Box

Explain why it exists, how it scales, and how it fails.

You are ready.